**HAKIZIMANA EMMANUJEL** **29/10/2025**
**REG NO: 224019436**
**UR/ACE-DS**
**Masters of Data Science in Mining**
**Module: ADVANCED DATABASE TECHNOLOGY**
**FINAL EXAM**

---

## SECTION A:

### QUESTION A1 SCREENSHOT

```
CREATE TABLE OrderDetail_A (
    OrderID NUMBER(10),
    ProductID NUMBER(10),
    Quantity NUMBER,
    SubTotal NUMBER(12, 2),
    CONSTRAINT PK_ODA PRIMARY KEY (OrderID, ProductID)
);

-- 2. Insert 5 rows into OrderDetail_A (Node A)
INSERT INTO OrderDetail_A (OrderID, ProductID, Quantity, SubTotal) VALUES (101, 1, 2, 20.00); -- Hash key 101
INSERT INTO OrderDetail_A (OrderID, ProductID, Quantity, SubTotal) VALUES (102, 2, 1, 15.50); -- Hash key 102
INSERT INTO OrderDetail_A (OrderID, ProductID, Quantity, SubTotal) VALUES (103, 1, 5, 50.00); -- Hash key 103
INSERT INTO OrderDetail_A (OrderID, ProductID, Quantity, SubTotal) VALUES (104, 3, 1, 8.25);  -- Hash key 104
INSERT INTO OrderDetail_A (OrderID, ProductID, Quantity, SubTotal) VALUES (105, 2, 3, 46.50); -- Hash key 105
COMMIT;
```

Script Output | Task completed in 0.259 seconds

```
Commit complete.
```

A1: OrderDetai_A table created in node A

```
CREATE TABLE OrderDetail_B (
    OrderID NUMBER(10),
    ProductID NUMBER(10),
    Quantity NUMBER,
    SubTotal NUMBER(12, 2),
    CONSTRAINT PK_ODB PRIMARY KEY (OrderID, ProductID)
);

-- 2. Insert 5 rows into OrderDetail_B (Node B)
INSERT INTO OrderDetail_B (OrderID, ProductID, Quantity, SubTotal) VALUES (201, 1, 4, 40.00); -- Hash key 201
INSERT INTO OrderDetail_B (OrderID, ProductID, Quantity, SubTotal) VALUES (202, 3, 2, 16.50); -- Hash key 202
INSERT INTO OrderDetail_B (OrderID, ProductID, Quantity, SubTotal) VALUES (203, 2, 1, 15.50); -- Hash key 203
INSERT INTO OrderDetail_B (OrderID, ProductID, Quantity, SubTotal) VALUES (204, 1, 1, 10.00); -- Hash key 204
INSERT INTO OrderDetail_B (OrderID, ProductID, Quantity, SubTotal) VALUES (205, 3, 3, 24.75); -- Hash key 205
COMMIT;
```

Script Output | Task completed in 0.102 seconds

```
Commit complete.
```

A1: OrderDetail_B table created in node B

```
-- Grant SELECT access on OrderDetail_B to Node A for the view
GRANT SELECT ON OrderDetail_B TO S_BranchDB_A;
```

Script Output

Task completed in 0.169 seconds

```
Grant succeeded.
```

A1: Grant to node A from node B succeeded

```
CREATE OR REPLACE VIEW OrderDetail_ALL AS
SELECT OrderID, ProductID, Quantity, SubTotal FROM OrderDetail_A       -- Local fragment
UNION ALL
SELECT OrderID, ProductID, Quantity, SubTotal FROM OrderDetail_B@PROJ_LINK; -- Remote fragment
```

Script Output

Task completed in 0.43 seconds

```
View ORDERDETAIL_ALL created.
```

A1: Created view ORDERDETAIL_ALL succeeded

```
SELECT
    COUNT(*) AS Fragment_Count,
    SUM(MOD(OrderID, 97)) AS Fragment_Checksum
FROM (
    SELECT OrderID FROM OrderDetail_A
    UNION ALL
    SELECT OrderID FROM OrderDetail_B@PROJ_LINK
);
-- EXPECTED OUTPUT: Fragment_Count = 10, Fragment_Checksum = (101+..+105 + 201+..+205) mod 97

SELECT
    COUNT(*) AS View_Count,
    SUM(MOD(OrderID, 97)) AS View_Checksum
FROM OrderDetail_ALL;
```

Query Result

SQL | All Rows Fetched: 1 in 0.011 seconds

| | VIEW_COUNT | VIEW_CHECKSUM |
|---|---|---|
| 1 | 10 | 75 |

A1: Validation output

## QUESTION A2 SCREENSHOTS

```oracle_table_ref
CREATE TABLE OrderInfo (
    OrderID NUMBER(10) PRIMARY KEY,
    CustomerID NUMBER,
    OrderDate DATE
);

INSERT INTO OrderInfo (OrderID, CustomerID, OrderDate) VALUES (101, 1, DATE '2025-10-01');
INSERT INTO OrderInfo (OrderID, CustomerID, OrderDate) VALUES (102, 2, DATE '2025-10-02');
INSERT INTO OrderInfo (OrderID, CustomerID, OrderDate) VALUES (201, 3, DATE '2025-10-03');
INSERT INTO OrderInfo (OrderID, CustomerID, OrderDate) VALUES (202, 4, DATE '2025-10-04');
INSERT INTO OrderInfo (OrderID, CustomerID, OrderDate) VALUES (300, 5, DATE '2025-10-05');
COMMIT;
CREATE TABLE Product (
    ProductID NUMBER(10) PRIMARY KEY,
    ProductName VARCHAR2(100),
    Category VARCHAR2(50)
);

INSERT INTO Product (ProductID, ProductName, Category) VALUES (1, 'Widget X', 'Electronics');
INSERT INTO Product (ProductID, ProductName, Category) VALUES (2, 'Widget Y', 'Electronics');
INSERT INTO Product (ProductID, ProductName, Category) VALUES (3, 'Gadget Z', 'Office');
INSERT INTO Product (ProductID, ProductName, Category) VALUES (4, 'Tool A', 'Office');
COMMIT;
```

Script Output

Task completed in 0.181 seconds

```
Commit complete.
```

A2: OrderInfo table well created in node B

```oracle_db_link
-- Create the database link 'PROJ_LINK' to Node B (Reusing link details from original code)
CREATE DATABASE LINK PROJ_LINK
CONNECT TO BranchDB_B IDENTIFIED BY "Emm788@enna"
USING '//Localhost:1521/XE';
```

Script Output

Task completed in 0.131 seconds

```
Database link PROJ_LINK created.
```

A2: Database link well created

```sql
SELECT
    ODA.OrderID,
    P.ProductName,
    ODA.Quantity,
    P.Category
FROM
    OrderDetail_A ODA                                   -- Local table (Node A)
JOIN
    Product@PROJ_LINK P ON ODA.ProductID = P.ProductID    -- Remote table (Node B)
WHERE
    ODA.ProductID IN (1, 3);
```

Query Result

SQL | All Rows Fetched: 0 in 0.031 seconds

| ORDERID | PRODUCT... | QUANTITY | CATEGORY |
|---|---|---|---|

A2: Row created

## QUESTION A3 SCREENSHOTS

```sql
    -- Enable AutoTrace and Timing
SET AUTOTRACE ON STATISTICS
SET TIMING ON

-- 1. SERIAL aggregation on OrderDetail_ALL (Local and Remote data)
SELECT
    ProductID,
    COUNT(*) AS TotalLines,
    SUM(Quantity) AS TotalQtySold,
    SUM(SubTotal) AS TotalRevenue
FROM
    OrderDetail_ALL
GROUP BY
    ProductID
ORDER BY
    TotalRevenue DESC;
```

Query Result

SQL | All Rows Fetched: 3 in 0.028 seconds

| | PRODUCTID | TOTALLINES | TOTALQTYSOLD | TOTALREVENUE |
|---|---|---|---|---|
| 1 | 1 | 4 | 15 | 124 |
| 2 | 2 | 3 | 5 | 77.5 |
| 3 | 3 | 3 | 6 | 49.5 |

A3: Serial Aggregation  on node A

```
SET AUTOTRACE ON STATISTICS
SET TIMING ON

SELECT /*+ GATHER_PLAN_STATISTICS */
       ProductID,
       COUNT(*) AS TotalLines,
       SUM(Quantity) AS TotalQtySold,
       SUM(SubTotal) AS TotalRevenue
FROM
    (SELECT /*+ PARALLEL(ODA, 8) */ OrderID, ProductID, Quantity, SubTotal FROM OrderDetail_A ODA
     UNION ALL
     SELECT OrderID, ProductID, Quantity, SubTotal FROM OrderDetail_B@PROJ_LINK)
GROUP BY ProductID
ORDER BY TotalRevenue DESC;
SET TIMING OFF
SET AUTOTRACE OFF
```

Query Result

SQL | All Rows Fetched: 3 in 0.026 seconds

| | PRODUCTID | TOTALLINES | TOTALQTYSOLD | TOTALREVENUE |
|---|---|---|---|---|
| 1 | 1 | 4 | 15 | 124 |
| 2 | 2 | 3 | 5 | 77.5 |
| 3 | 3 | 3 | 6 | 49.5 |

A3: Parallel Aggregation

```
EXPLAIN PLAN FOR
SELECT ProductID, COUNT(*), SUM(Quantity), SUM(SubTotal) FROM OrderDetail_ALL GROUP BY ProductID;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- Check for 'REMOTE' operation (for OrderDetail_B) but no 'PX' (Parallel) steps.

--4 Parallel Plan (Using the specific parallel hint)
EXPLAIN PLAN FOR
SELECT /*+ PARALLEL(OrderDetail_A, 8) */ ProductID, COUNT(*), SUM(Quantity), SUM(SubTotal)
FROM (
    SELECT * FROM OrderDetail_A
    UNION ALL
    SELECT * FROM OrderDetail_B@PROJ_LINK
)
GROUP BY ProductID;
```

Script Output

Task completed in 0.137 seconds

```
Explained.
```

A3: Explain Plan well explained

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Script Output | Query Result

All Rows Fetched: 26 in 0.281 seconds

PLAN_TABLE_OUTPUT

```
---------------------------------------------------------------------------------------------------
| Id  | Operation            | Name           | Rows  | Bytes | Cost (%CPU)| Time     | Inst   |IN-OUT|
---------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT     |                |     3 |    84 |     7  (15)| 00:00:01 |        |      |
|   1 |  HASH GROUP BY       |                |     3 |    84 |     7  (15)| 00:00:01 |        |      |
|   2 |   VIEW               |                |    10 |   280 |     6   (0)| 00:00:01 |        |      |
|   3 |    UNION-ALL         |                |       |       |            |          |        |      |
|   4 |     TABLE ACCESS FULL| ORDERDETAIL_A  |     5 |    50 |     3   (0)| 00:00:01 |        |      |
|   5 |     REMOTE           | ORDERDETAIL_B  |     5 |    70 |     3   (0)| 00:00:01 | PROJ_~ | R->S |
---------------------------------------------------------------------------------------------------

Remote SQL Information (identified by operation id):
----------------------------------------------------

   5 - SELECT "PRODUCTID","QUANTITY","SUBTOTAL" FROM "ORDERDETAIL_B" "ORDERDETAIL_B"
       (accessing 'PROJ_LINK' )


Hint Report (identified by operation id / Query Block Name / Object Alias):
Total hints for statement: 1 (N - Unresolved (1))
---------------------------------------------------------------------------

   1 -  SEL$1
         N -  PARALLEL(OrderDetail_A, 8)
```

A3: Explained Plan Table

## QUESTION A3 SCREENSHOTS

```
-- 1.Clean up any test rows from A2 if OrderInfo wasn't minimal
DELETE FROM OrderInfo_KGL WHERE OrderID IN (1000, 1001);
DELETE FROM OrderInfo@PROJ_LINK WHERE OrderID IN (1000, 1001);
COMMIT;
```

Script Output

Task completed in 0.083 seconds

```
Commit complete.
```

A3: Transaction on both node committed successful

N.B: Oracle don't support concurrent transaction failure ( see the error below):

```
BEGIN
    -- Insert into local node (Kigali)
    INSERT INTO OrderInfo_KGL (OrderID, CustomerID, OrderDate)
    VALUES (1000, 10, DATE '2025-10-28');
    -- Insert into remote node (Other cities)
    INSERT INTO OrderInfo@PROJ_LINK (OrderID, CustomerID, OrderDate)
    VALUES (1001, 11, DATE '2025-10-28');
     -- Single commit triggers Two-Phase Commit (2PC)
    COMMIT;
```

Script Output | Task completed in 0.137 seconds

```
PL/SQL procedure successfully completed.
```

A4: Inserting into local and remote node successfully completed

```
--      DBMS_OUTPUT.PUT_LINE('Clean 2PC completed successfully');
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
--        DBMS_OUTPUT.PUT_LINE('2PC failed and rolled back: ' || SQLERRM);
END;
/
```

Script Output | Task completed in 0.167 seconds

```
Error starting at line : 128 in command -
    WHEN OTHERS THEN
Error report -
Unknown Command


Rollback complete.


Error starting at line : 131 in command -
END
Error report -
Unknown Command

SP2-0044: For a list of known commands enter HELP
and to leave enter EXIT.
```

A4: Roll back completed with error

```
-- Verify committed data (should see both 1000 and 1001)
SELECT 'Local' AS Node, OrderID FROM OrderInfo_KGL WHERE OrderID = 1000
UNION ALL
SELECT 'Remote' AS Node, OrderID FROM OrderInfo@PROJ_LINK WHERE OrderID = 1001;
```

Query Result | SQL | All Rows Fetched: 3 in 0.022 seconds

| | NODE | ORDERID |
|---|---|---|
| 1 | Local | 1000 |
| 2 | Local | 1000 |
| 3 | Remote | 1001 |

A: Verification of Transaction made before ( succeeded )

```
BEGIN
    -- Insert into local node (Kigali)
    INSERT INTO OrderInfo_KGL (OrderID, CustomerID, OrderDate)
    VALUES (1000, 10, DATE '2025-10-28');
    -- Insert into remote node (Other cities)
    INSERT INTO OrderInfo@PROJ_LINK (OrderID, CustomerID, OrderDate)
    VALUES (1001, 11, DATE '2025-10-28');
    -- When COMMIT is hit, the transaction becomes in-doubt (prepared locally, but commit failed remotely)
    COMMIT;
```

Script Output

Task completed in 0.153 seconds

```
    INSERT INTO OrderInfo_KGL (OrderID, CustomerID, OrderDate)
                *
ERROR at line 3:
ORA-06550: line 3, column 17:
PL/SQL: ORA-00942: table or view does not exist
ORA-06550: line 3, column 5:
PL/SQL: SQL Statement ignored
ORA-06550: line 6, column 5:
PL/SQL: ORA-00942: table or view does not exist
ORA-06550: line 6, column 5:
PL/SQL: SQL Statement ignored

https://docs.oracle.com/error-help/db/ora-06550/


More Details :
https://docs.oracle.com/error-help/db/ora-06550/
```

A4: This was run in remote node and it shows that oracle don't support failure transaction

```
BEGIN
    -- Insert into local node (Kigali)
    INSERT INTO OrderInfo_KGL (OrderID, CustomerID, OrderDate)
    VALUES (1000, 10, DATE '2025-10-28');
    -- Insert into remote node (Other cities)
    INSERT INTO OrderInfo@PROJ_LINK (OrderID, CustomerID, OrderDate)
    VALUES (1001, 11, DATE '2025-10-28');
    -- When COMMIT is hit, the transaction becomes in-doubt (prepared locally, but commit failed remotely)
    COMMIT;

    -- This may fail with ORA-02055/ORA-02063
```

Script Output

Task completed in 0.102 seconds

```
PL/SQL procedure successfully completed.
```

A4: It is again run in local node, and automatically succeeded ( didn't fail)

```
-- 4. Re-verify consistency (as S_BranchDB_A)
SELECT 'Local' AS Node, OrderID FROM OrderInfo_KGL WHERE OrderID = 1000
UNION ALL
SELECT 'Remote' AS Node, OrderID FROM OrderInfo@PROJ_LINK WHERE OrderID = 1001;
-- Output should show 0 rows if ROLLBACK FORCE succeeded
```

Query Result

SQL | All Rows Fetched: 4 in 0.007 seconds

| | NODE | ORDERID |
|---|---|---|
| 1 | Local | 1000 |
| 2 | Local | 1000 |
| 3 | Local | 1000 |
| 4 | Remote | 1001 |

A4: Consistency Verification output on both node .

## QUESTION A5 SCREENSHOTS

```
-- 1. UPDATE a single remote row and keep the transaction open (no COMMIT)
UPDATE OrderDetail_B@PROJ_LINK
SET SubTotal = SubTotal * 1.1
WHERE OrderID = 201 AND ProductID = 1;
```

Script Output

Task completed in 0.069 seconds

```
1 row updated.
```

A5: Single row well updated

```
UPDATE OrderDetail_B
SET Quantity = Quantity + 1
WHERE OrderID = 201 AND ProductID = 1;
```

Script Output

ScriptRunner Task

A: WAITER SESSION IN NODE B

```
SELECT
    L.SID AS WAITER_SID,
    S.SERIAL# AS WAITER_SERIAL,
    L.BLOCKING_SESSION AS BLOCKER_SID,
    BS.SERIAL# AS BLOCKER_SERIAL,
    S.EVENT,
    'Contesting Row: 201/1' AS CONTEXT
FROM
    V$SESSION S
JOIN
    V$SESSION BS ON S.BLOCKING_SESSION = BS.SID
JOIN
    V$SESSION_WAIT W ON S.SID = W.SID
JOIN
    V$LOCK L ON S.SID = L.SID
WHERE
    W.EVENT LIKE 'SQL*Net message from dblink%' -- Waiting on a distributed transaction
    OR S.STATE = 'WAITING' AND S.ROW_WAIT_OBJ# IS NOT NULL;
--

-- 4. Release the lock by committing Session 1
COMMIT;
```

Script Output

Task completed in 0.265 seconds

```
Commit complete.
```

A5: After committing the waiter session automatically runnel well as shown below

```
UPDATE OrderDetail_B
SET Quantity = Quantity + 1
WHERE OrderID = 201 AND ProductID = 1;
```

Script Output

Task completed in 867.723 seconds

```
1 row updated.
```

A5: Waiter session now completed

```
SELECT * FROM OrderDetail_B@PROJ_LINK WHERE OrderID = 201 AND ProductID = 1;
```

Query Result

SQL | All Rows Fetched: 1 in 0.072 seconds

| | ORDERID | PRODUCTID | QUANTITY | SUBTOTAL |
|---|---|---|---|---|
| 1 | 201 | 1 | 7 | 53.24 |

A5: Data verification in local node